

DEEP LEARNING

PROFESSIONAL HANDBOOK

Beginner to Intermediate

Concepts • Mathematics • Training • Architectures • Deployment

Learning promise: This handbook assumes you are new to Deep Learning. It explains the ideas from first principles, then builds toward intermediate topics such as optimization, convolutional networks, sequence models, attention, Transformers, transfer learning, debugging, and production inference.

Recommended background: basic Python, basic algebra, and a beginner-level understanding of machine learning. You do not need prior neural-network knowledge.

How to use this handbook

- Read Chapters 1-10 in order if you are learning Deep Learning for the first time.
- Run the code examples instead of only reading them. Change values and observe what happens.
- Do not memorize formulas. Focus first on what each formula is trying to accomplish.
- After the architecture chapters, build one small project and debug it using the workflow in the final chapters.

Contents and Learning Path

#	Chapter	What you will learn
1	What Deep Learning Is	The big picture, ML vs DL, terminology
2	From Neuron to Neural Network	Perceptrons, layers, parameters, tensors
3	Forward Propagation	How data becomes predictions
4	Activation Functions	ReLU, sigmoid, tanh, softmax, GELU
5	Loss Functions	What the model is trying to minimize
6	Backpropagation	Gradients and the chain rule
7	Optimizers	SGD, Momentum, RMSProp, Adam
8	Initialization, Normalization and Regularization	Stable training and generalization
9	A Complete Training Pipeline	Data → model → train → validate → save
10	Multilayer Perceptrons	Deep fully connected networks
11	Convolutional Neural Networks	Images and spatial patterns
12	RNN, LSTM and GRU	Sequential data and memory
13	Attention and Transformers	Modern sequence modeling
14	Transfer Learning and Fine-Tuning	Reuse pretrained models
15	Evaluation and Debugging	Metrics, curves, failure diagnosis
16	Deployment and Live Inference	Saving, loading, APIs, drift
17	Intermediate Practice Projects	How to turn knowledge into skill
A	Cheat Sheets and Glossary	Fast reference for revision

Target level after completion: You should be able to explain how neural networks learn, choose a reasonable architecture and loss function, train and evaluate a model, understand common training failures, use transfer learning, and deploy a saved model for inference. That is a practical intermediate foundation - not expert research-level mastery.

1. What Deep Learning Is

Start with the mental model before touching code.

1.1 Machine Learning vs Deep Learning

Machine Learning is a broad field where algorithms learn patterns from data. Deep Learning is a subfield of Machine Learning that uses neural networks with many layers to learn increasingly useful representations from data.

Traditional ML	Deep Learning
----------------	---------------

Often depends heavily on manually designed features.	Can learn useful features automatically from raw or lightly processed data.
Works very well on many structured/tabular datasets.	Excels especially on images, audio, text, video and large complex datasets.
Models may be easier to interpret.	Models can have millions or billions of parameters and may be harder to interpret.
Usually needs less compute.	Often benefits from GPUs/TPUs and more data.

Mental model: A deep network is a function with many adjustable parameters. Training repeatedly changes those parameters so the function produces outputs that better match the desired targets.

1.2 Why “deep”?

A neural network is called deep when it contains multiple layers between the input and the final output. Each layer transforms the representation. Early layers may learn simple patterns; later layers can combine them into more abstract patterns.

Example for an image classifier: pixels → edges → textures → object parts → object category.

1.3 Core vocabulary

Term	Meaning
Feature	Input information used by a model.
Label / target	The answer the model is expected to learn.
Sample	One data example.
Batch	A group of samples processed together.
Epoch	One full pass through the training dataset.
Parameter	A value learned by the model, such as a weight or bias.
Hyperparameter	A setting chosen by you, such as learning rate, batch size or number of layers.
Inference	Using a trained model to make predictions on new data.
Gradient	Information describing how a parameter should change to affect the loss.

2. From a Neuron to a Neural Network

Understand the smallest computational unit and how networks are assembled.

2.1 The artificial neuron

A neuron receives one or more inputs, multiplies each input by a weight, adds a bias, and then passes the result through an activation function.

Core equation: $z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$, followed by $a = \text{activation}(z)$.

The weights determine how strongly each input influences the neuron. The bias shifts the neuron's response. The activation function allows the network to represent non-linear relationships.

2.2 A tiny numerical example

Suppose $x_1=2$, $x_2=3$, $w_1=0.5$, $w_2=-0.2$, $b=0.1$. Then $z=(2 \times 0.5) + (3 \times -0.2) + 0.1 = 0.5$. If the activation is ReLU, the output is $\max(0, 0.5) = 0.5$.

2.3 Layers

- Input layer: receives features.
- Hidden layers: transform representations.
- Output layer: produces the prediction.

A fully connected (Dense) layer connects every neuron in one layer to every neuron in the next layer. Convolutional and recurrent layers use more specialized connectivity.

2.4 Parameters grow quickly

For a Dense layer with 100 input features and 64 neurons, the number of trainable parameters is 100×64 weights + 64 biases = 6,464 parameters.

Why this matters: Model capacity grows with parameters. More capacity can learn more complex patterns, but it also increases compute cost and the risk of overfitting.

2.5 Tensors

Deep-learning frameworks represent data with tensors. A tensor is a multidimensional array.

Rank	Example	Typical meaning
0	scalar	one number
1	[f1,f2,...]	feature vector
2	rows $\times$ columns	table or batch of vectors
3	height $\times$ width $\times$ channels	single image, depending on convention
4	batch $\times$ height $\times$ width $\times$ channels	batch of images

3. Forward Propagation

See exactly how a network turns input into output.

3.1 The forward pass

Forward propagation is the process of sending input through every layer to produce a prediction. No parameters are changed during the forward pass itself.

1. Read the input batch.
2. Compute weighted sums in the first layer.
3. Apply an activation function.
4. Pass the resulting activations into the next layer.
5. Continue until the output layer produces predictions.

3.2 Matrix form

For a Dense layer, a whole batch can be processed efficiently using matrix multiplication: $Z = XW + b$, then $A = \text{activation}(Z)$. GPUs are very effective at this kind of large parallel numerical computation.

3.3 Output layers depend on the task

Task	Typical output	Typical activation
Regression	One or more real numbers	Linear / no activation
Binary classification	One probability	Sigmoid
Multiclass classification	One probability per class	Softmax
Multi-label classification	Independent probability per label	Sigmoid

4. Activation Functions

Activation functions give neural networks the ability to learn non-linear patterns.

4.1 Why non-linearity is necessary

If every layer performed only linear operations, stacking many layers would still be equivalent to a single linear transformation. Non-linear activations let deep networks approximate complex functions.

4.2 ReLU

$\text{ReLU}(x) = \max(0, x)$. It is the most common default activation for hidden layers in many feed-forward and convolutional networks because it is simple and usually trains efficiently.

Weakness: neurons can become inactive if their inputs remain negative. Variants such as Leaky ReLU reduce this problem.

4.3 Sigmoid

$\text{Sigmoid}(x) = 1/(1 + e^{-x})$, producing values between 0 and 1. It is useful for binary output probabilities, but is usually not the best hidden-layer activation because gradients can become very small at extreme inputs.

4.4 Tanh

Tanh produces values between -1 and 1. It is zero-centered and historically common in recurrent networks, although modern architectures often use other choices internally.

4.5 Softmax

Softmax converts a vector of logits into a probability distribution whose values sum to 1. It is typically used in single-label multiclass classification.

4.6 GELU

GELU is common in Transformer architectures. It behaves like a smooth, probabilistic gating function and works well in large modern networks.

Activation	Typical use	Main caution
ReLU	Hidden layers	Dead neurons possible

Sigmoid	Binary output	Saturating gradients
Tanh	Some recurrent components	Saturation
Softmax	Multiclass output	Classes treated as mutually exclusive
GELU	Transformers	Slightly more computationally complex

5. Loss Functions

The loss tells the model how wrong it is.

5.1 Loss vs metric

The loss is the differentiable objective optimized during training. A metric is a quantity used to judge performance. Sometimes they are related, but they do not have to be the same.

5.2 Regression losses

Mean Squared Error (MSE) averages squared prediction errors. It strongly penalizes large mistakes. Mean Absolute Error (MAE) is more robust to large outliers because errors are not squared.

5.3 Binary cross-entropy

Binary cross-entropy measures how well predicted probabilities match binary labels. It strongly penalizes confident wrong predictions.

5.4 Categorical cross-entropy

Categorical cross-entropy is widely used when exactly one class is correct among multiple classes. Sparse categorical cross-entropy is convenient when labels are stored as integer class IDs instead of one-hot vectors.

Rule of thumb: Choose the output layer and loss function together. Binary: sigmoid + binary cross-entropy. Multiclass single-label: softmax + categorical/sparse categorical cross-entropy. Regression: linear output + MSE/MAE/Huber depending on the problem.

6. Backpropagation

Backpropagation is how the network learns which parameters caused the error.

6.1 The idea

After a forward pass, the network has a prediction and a loss. Backpropagation computes the gradient of that loss with respect to every trainable parameter. The optimizer then uses those gradients to update the parameters.

6.2 The chain rule

A deep network is a composition of functions. The chain rule allows us to determine how a small change in an earlier parameter influences the final loss through all later operations.

Conceptual chain: parameter → neuron output → next layer → prediction → loss. Backpropagation works backward through this chain.

6.3 Gradient interpretation

If the gradient of the loss with respect to a weight is positive, increasing that weight would increase the loss locally, so gradient descent tends to reduce the weight. If the gradient is negative, the update tends to increase the weight.

6.4 Automatic differentiation

Modern frameworks such as TensorFlow and PyTorch automatically record differentiable operations and compute gradients. You still need to understand the concept because many training failures - exploding gradients, vanishing gradients, bad learning rates - are gradient problems.

6.5 Vanishing and exploding gradients

Vanishing gradients become extremely small, making early layers learn slowly. Exploding gradients become very large, making training unstable. Good initialization, ReLU-like activations, normalization, residual connections, gated recurrent units and gradient clipping are common remedies.

7. Optimizers

Optimizers decide how parameters change after gradients are computed.

7.1 Gradient descent

The basic update rule is: $\text{parameter} = \text{parameter} - \text{learning_rate} \times \text{gradient}$. The learning rate controls the step size.

7.2 Stochastic and mini-batch training

Full-batch gradient descent uses the entire dataset for each update. Stochastic gradient descent uses one sample. In practice, mini-batch training is the standard compromise: efficient on hardware, less memory-intensive, and introduces useful gradient noise.

7.3 SGD with momentum

Momentum accumulates a moving direction of past gradients. It can accelerate movement along consistent directions and reduce oscillation.

7.4 RMSProp

RMSProp adapts the effective learning rate per parameter using a moving average of squared gradients. It historically performed well for recurrent networks.

7.5 Adam

Adam combines momentum-like first-moment estimates with adaptive second-moment scaling. It is a strong default for many problems because it often trains quickly without extensive tuning.

Optimizer	Good starting use	Watch for
SGD	Strong final generalization, classic CNN training	Needs learning-rate tuning
SGD + Momentum	Stable directional updates	Still sensitive to schedule
RMSProp	Some sequence/recurrent tasks	Less common as general default
Adam	General-purpose default	Can overfit or settle differently than SGD

7.6 Learning-rate schedules

A fixed learning rate is not always ideal. Common schedules reduce it over time, after validation plateaus, or according to cosine decay. Warm-up is often used in large Transformer training.

8. Initialization, Normalization and Regularization

These techniques often decide whether a network trains smoothly and generalizes.

8.1 Weight initialization

Initializing every weight to zero would make neurons learn identical features. Random initialization breaks symmetry, but the scale matters. Xavier/Glorot initialization is common for tanh-like activations; He initialization is common for ReLU-like activations.

8.2 Batch Normalization

Batch Normalization normalizes intermediate activations using batch statistics and learns scale/shift parameters. It can make training faster and more stable and sometimes adds regularization-like noise.

8.3 Layer Normalization

Layer Normalization normalizes across features within each sample rather than across the batch. It is a standard component of Transformer architectures.

8.4 Dropout

During training, Dropout randomly sets a fraction of activations to zero. This discourages units from depending too strongly on specific partners and can reduce overfitting. Dropout is disabled during inference.

8.5 Weight decay / L2 regularization

Weight decay discourages very large parameter values. In modern optimizers, decoupled weight decay such as AdamW is often preferred when appropriate.

8.6 Early stopping

Early stopping monitors validation performance and stops training when generalization no longer improves. It is one of the easiest practical defenses against overfitting.

Important: Regularization should solve a real generalization problem. Do not add every technique automatically. Start with a baseline, inspect training/validation behavior, then add the smallest intervention that addresses the observed failure.

9. A Complete Training Pipeline

This chapter connects the theory into a real working workflow.

9.1 Standard workflow

6. Define the task and metric.
7. Collect and inspect data.
8. Split into train, validation and test sets.
9. Build preprocessing using training data only.
10. Create a baseline architecture.

11. Choose loss, optimizer and metrics.
12. Train while monitoring validation performance.
13. Tune using the validation set.
14. Evaluate once on the held-out test set.
15. Save preprocessing + model together for deployment.

9.2 Data split discipline

The training set is used to learn parameters. The validation set is used to tune architecture and hyperparameters. The test set should represent a final unbiased check and should not become a hidden validation set through repeated tuning.

9.3 First complete Keras example - binary classification

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# X: numerical features, y: 0/1 labels
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

model = keras.Sequential([
    keras.layers.Input(shape=(X_train.shape[1],)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.25),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=5, restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

print(model.evaluate(X_test, y_test, verbose=0))
model.save("binary_classifier.keras")
```

9.4 What to watch while training

Pattern	Likely meaning	Typical response
Train loss decreases; val loss also decreases	Healthy learning	Continue until improvement slows
Train improves; validation worsens	Overfitting	Regularize, add data, simplify, early stop
Both losses stay high	Underfitting or optimization issue	Increase capacity, train longer, improve features, tune LR
Loss becomes NaN	Numerical instability	Lower LR, normalize, inspect data, clip gradients
Accuracy high but recall poor	Class imbalance / threshold issue	Use better metrics, class weights, threshold tuning

10. Multilayer Perceptrons (MLPs)

The standard fully connected deep network and the best place to understand core DL mechanics.

10.1 What an MLP is

An MLP stacks Dense layers and non-linear activations. It is appropriate for many tabular or vectorized inputs, although tree-based ML models often remain very competitive on structured tabular data.

10.2 Designing depth and width

Width means neurons per layer; depth means number of layers. More is not automatically better. A useful baseline might have 2-4 hidden layers with widths that gradually decrease, but architecture should follow problem complexity and data volume.

10.3 Multiclass example

```

model = keras.Sequential([
    keras.layers.Input(shape=(20,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.30),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(5, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

10.4 Intermediate concept: logits

A logit is the raw output score before a probability activation such as sigmoid or softmax. Some loss implementations combine the activation and loss numerically for stability. When using such APIs, make sure the model output and the loss configuration agree on whether logits or probabilities are being supplied.

11. Convolutional Neural Networks (CNNs)

CNNs exploit spatial structure and are foundational for computer vision.

11.1 Why Dense layers are inefficient for images

A 224×224 RGB image contains 150,528 input values. Fully connecting that directly to thousands of neurons creates huge parameter counts and ignores locality. CNNs use small filters that scan across the image and reuse the same weights at many positions.

11.2 Convolution

A convolutional filter is a small learnable kernel, such as 3×3. It slides across the image and computes local weighted combinations. Different filters can learn different patterns such as edges, textures and shapes.

11.3 Important terms

Term	Meaning
Kernel/filter	Small learnable weight matrix
Feature map	Output produced by applying a filter
Stride	How far the filter moves each step
Padding	Extra border values used to control spatial size
Pooling	Downsampling operation
Channels	Feature dimensions; RGB images have 3 input channels

11.4 CNN example

```
model = keras.Sequential([
    keras.layers.Input(shape=(128, 128, 3)),
    keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    keras.layers.MaxPooling2D(),
    keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    keras.layers.MaxPooling2D(),
    keras.layers.Conv2D(128, 3, padding="same", activation="relu"),
    keras.layers.GlobalAveragePooling2D(),
    keras.layers.Dropout(0.30),
    keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

11.5 Data augmentation

Data augmentation creates realistic training variations such as flips, crops, rotations or brightness changes. It can improve invariance and reduce overfitting. Augmentations must preserve the label semantics; for example, flipping some medical images may not be appropriate.

11.6 Residual connections

Residual networks add shortcut connections that let information and gradients bypass layers. Instead of learning a complete transformation $H(x)$, a block can learn a residual $F(x)$ and output $x + F(x)$. This made very deep networks much easier to optimize.

12. RNN, LSTM and GRU

Sequence models process ordered information such as time series, text or sensor streams.

12.1 Recurrent Neural Networks

An RNN processes a sequence one step at a time while carrying a hidden state that summarizes previous steps. The same recurrent parameters are reused across time.

12.2 Why ordinary RNNs struggle

When sequences are long, repeated multiplication during backpropagation through time can cause gradients to vanish or explode. This makes it difficult to learn long-range dependencies.

12.3 LSTM

LSTM introduces a cell state and gates that decide what information to forget, write and expose. These gates create paths where useful information and gradients can persist longer.

12.4 GRU

GRU is a simpler gated recurrent architecture with fewer gates and parameters than LSTM. It often performs similarly and can be faster.

12.5 Sequence example

```
model = keras.Sequential([
    keras.layers.Input(shape=(100, 32)), # 100 time steps, 32 features
    keras.layers.Bidirectional(keras.layers.LSTM(64)),
    keras.layers.Dropout(0.30),
    keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

12.6 When to use recurrent models today

RNN/LSTM/GRU models remain useful for many time-series and smaller sequence problems. For large-scale NLP and many long-context sequence tasks, Transformer-based models are now more common because they parallelize better and capture long-range dependencies more directly.

13. Attention and Transformers

This chapter introduces the architecture behind modern large language models and many vision models.

13.1 The intuition of attention

Attention lets a model decide which other positions are most relevant when processing a particular position. Instead of compressing an entire sequence into one fixed recurrent state, the model can directly combine information from many positions.

13.2 Query, Key and Value

Each token representation is projected into three vectors: Query (what am I looking for?), Key (what information do I contain?), and Value (what content should be passed forward?). Similarity between queries and keys determines how strongly values are combined.

Scaled dot-product attention: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$. The scaling term helps keep dot products numerically well-behaved as dimensionality grows.

13.3 Multi-head attention

Instead of performing one attention operation, multi-head attention uses several learned projections in parallel. Different heads can learn different relationships, such as local syntax, long-distance references or semantic associations.

13.4 Positional information

Attention alone does not inherently know token order. Transformers therefore inject positional information using positional encodings or learned/relative position mechanisms.

13.5 Transformer block

- Multi-head self-attention
- Residual connection + normalization
- Feed-forward network
- Residual connection + normalization

Modern implementations vary in ordering, normalization style and positional mechanisms, but this is the core idea.

13.6 Encoder vs decoder

Architecture	Typical purpose	Attention pattern
Encoder	Understand an input representation	Usually bidirectional self-attention
Decoder	Generate output autoregressively	Masked causal self-attention
Encoder-decoder	Transform one sequence into another	Encoder self-attention + decoder self-attention + cross-attention

13.7 Tiny Keras attention block

```
inputs = keras.Input(shape=(100, 128))

attn = keras.layers.MultiHeadAttention(
    num_heads=4, key_dim=32
)(inputs, inputs)

x = keras.layers.LayerNormalization()(inputs + attn)
ff = keras.layers.Dense(256, activation="gelu")(x)
```

```
ff = keras.layers.Dense(128)(ff)
x = keras.layers.LayerNormalization()(x + ff)

model = keras.Model(inputs, x)
```

13.8 Intermediate understanding

Transformers trade the sequential recurrence of RNNs for attention-based interaction. Training can be highly parallel, but vanilla self-attention has quadratic cost with sequence length because every position may attend to every other position.

14. Transfer Learning and Fine-Tuning

Use knowledge learned on large datasets instead of training everything from scratch.

14.1 Transfer learning

A pretrained model has already learned useful representations. You can reuse its feature extractor and train a smaller task-specific head on your dataset. This is especially valuable when your labeled dataset is limited.

14.2 Standard workflow

16. Load a pretrained backbone.
17. Freeze most or all pretrained layers.
18. Train a new task-specific output head.
19. Unfreeze some upper layers.
20. Fine-tune with a much smaller learning rate.
21. Monitor validation performance closely.

14.3 Image example with MobileNetV2

```
base = keras.applications.MobileNetV2(
    include_top=False,
    weights="imagenet",
    input_shape=(224, 224, 3)
)
base.trainable = False

inputs = keras.Input(shape=(224, 224, 3))
x = keras.applications.mobilenet_v2.preprocess_input(inputs)
x = base(x, training=False)
x = keras.layers.GlobalAveragePooling2D()(x)
x = keras.layers.Dropout(0.2)(x)
outputs = keras.layers.Dense(5, activation="softmax")(x)

model = keras.Model(inputs, outputs)
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

14.4 Fine-tuning carefully

After the new head learns reasonable weights, you can unfreeze a portion of the backbone. Use a small learning rate so you do not destroy useful pretrained representations. This destructive change is sometimes called catastrophic forgetting.

15. Evaluation and Debugging

Good Deep Learning is not only building networks; it is diagnosing what the network is doing.

15.1 Classification metrics

Metric	What it answers
Accuracy	How many predictions were correct overall?
Precision	When the model predicts positive, how often is it correct?
Recall	Of all real positives, how many did it find?
F1	How well are precision and recall balanced?
ROC-AUC	How well does the model rank positive above negative across thresholds?
PR-AUC	Useful ranking summary when positive cases are rare

15.2 Confusion matrix

A confusion matrix separates true positives, true negatives, false positives and false negatives. This is often much more informative than one overall accuracy value.

15.3 Regression metrics

MAE is easy to interpret in target units. RMSE penalizes larger errors more strongly. R^2 describes explained variance but should not be treated as an “accuracy percentage.”

15.4 Learning curves

Plot training and validation loss across epochs. The gap and direction often reveal underfitting, overfitting or optimization problems faster than the final score alone.

15.5 Debugging checklist

- Can the model overfit a tiny sample? If not, there may be a bug in data, labels, loss or architecture.
- Are inputs scaled or normalized as expected?
- Does the output activation match the labels and loss?
- Is class imbalance hiding behind accuracy?
- Is the learning rate too large or too small?
- Are training and validation distributions meaningfully similar?
- Did preprocessing leak information from validation/test data?
- Are labels noisy, incorrect or misaligned?
- Does validation performance improve when you simplify the model?

15.6 Reproducibility

Set random seeds when you need repeatable experiments, but understand that some GPU operations may still be nondeterministic depending on the framework and hardware. Log model configuration, preprocessing, dataset version and metrics for every serious experiment.

16. Deployment and Live Inference

Training and deployment are different programs with different responsibilities.

16.1 Training side

Training uses historical data, performs gradient updates, validates performance and saves a model artifact. It may run for minutes, hours or days.

16.2 Inference side

Inference loads the saved model, applies the exact same preprocessing to new input, runs a forward pass and returns a prediction. It should not retrain the model for each user request.

16.3 Save and load

```
# Training
model.save("my_model.keras")

# Deployment / inference
from tensorflow import keras
model = keras.models.load_model("my_model.keras")
prediction = model.predict(new_input, verbose=0)
```

16.4 Simple API shape

```
# Example structure using FastAPI
from fastapi import FastAPI
from tensorflow import keras
import numpy as np

app = FastAPI()
model = keras.models.load_model("my_model.keras")

@app.post("/predict")
def predict(features: list[float]):
    x = np.array([features], dtype=np.float32)
    # Apply the SAME preprocessing used during training here.
    prob = float(model.predict(x, verbose=0)[0][0])
    return {"probability": prob, "class": int(prob >= 0.5)}
```

16.5 Production concerns

Concern	Why it matters
Preprocessing parity	Training and serving must transform data identically.
Latency	Users may need predictions in milliseconds.
Throughput	Batching can improve hardware efficiency.
Versioning	You need to know exactly which model produced a prediction.
Monitoring	Input distributions and performance can drift.
Fallbacks	A model service can fail; production systems need safe behavior.

16.6 Model drift

Data drift means input data changes. Concept drift means the relationship between input and target changes. A deployed model may therefore degrade even if the code never changes. Monitoring and periodic re-evaluation are part of the ML lifecycle.

17. Intermediate Practice Projects

The fastest path from reading to competence is deliberate implementation.

17.1 Project 1 - Tabular binary classifier

Build an MLP for churn, purchase prediction or risk classification. Compare it against Logistic Regression and a tree ensemble. This teaches you that Deep Learning is not automatically the best choice for every dataset.

17.2 Project 2 - Image classifier

Train a small CNN, then solve the same task with transfer learning. Compare training time, accuracy, overfitting and data requirements.

17.3 Project 3 - Time-series classifier or forecaster

Use a sliding window to create sequences, then compare an MLP, LSTM/GRU and a simple baseline. Learn why data splitting by time is different from random splitting.

17.4 Project 4 - Text classification

Start with tokenization and embeddings, then try a recurrent model or a pretrained Transformer. Measure not just accuracy but class-wise precision/recall.

17.5 What you should document

- Problem definition and success metric
- Dataset description and leakage risks
- Baseline
- Architecture and parameter count
- Training settings
- Learning curves
- Final validation/test metrics
- Error analysis
- Deployment input/output contract
- Known limitations

Intermediate-level milestone: You are not done when the model trains. You are intermediate when you can explain why you chose the architecture, detect overfitting/underfitting, select suitable metrics, debug a failed run, compare against a baseline, and deploy inference reproducibly.

Appendix A. Cheat Sheets and Glossary

A compact reference to revise after you understand the chapters.

A.1 Task → output → loss cheat sheet

Problem	Output layer	Typical loss
Regression	Linear	MSE / MAE / Huber
Binary classification	1 sigmoid unit	Binary cross-entropy
Multiclass single-label	N softmax units	Sparse/categorical cross-entropy
Multi-label	N sigmoid units	Binary cross-entropy

A.2 Training problems cheat sheet

Symptom	Possible cause	First things to try
Loss does not decrease	Bad LR, wrong labels/loss, broken pipeline	Inspect batch, lower/raise LR, tiny-set overfit test
Train good, val bad	Overfitting	More data, augmentation, regularization, early stopping
Both poor	Underfitting	More capacity, better features, longer training
NaN loss	Numerical instability	Lower LR, normalize inputs, inspect extreme values
Predictions all one class	Imbalance, threshold, output/loss mismatch	Check label distribution and probabilities

A.3 Essential glossary

Term	Meaning
Activation	Non-linear function applied after a layer transformation.
Backpropagation	Efficient gradient computation through a computational graph.
Batch	Group of samples used for one forward/backward update.
Epoch	One pass through the training dataset.
Embedding	Dense learned vector representation of discrete items such as tokens.
Fine-tuning	Continuing training of a pretrained model on a target task.
Gradient	Derivative of the loss with respect to a parameter.
Inference	Prediction using trained parameters without gradient updates.
Learning rate	Step-size scale used by an optimizer.
Logit	Raw prediction score before a probability activation.

Overfitting	Model performs much better on training data than unseen data.
Parameter	Learned weight or bias.
Regularization	Technique used to improve generalization.
Representation	Internal feature encoding learned by the network.
Tensor	Multidimensional numerical array.
Transfer learning	Reusing knowledge from a pretrained model.

A.4 Practical default starting points

- Hidden activations: ReLU for standard MLP/CNN baselines.
- Optimizer: Adam is a reasonable first baseline; tune learning rate before changing many other things.
- Batch size: 32 or 64 is a reasonable starting point for many small projects, but hardware and task matter.
- Validation: monitor validation loss, not only training accuracy.
- Regularization: early stopping is often the first technique to add; then consider dropout, augmentation or weight decay based on the problem.
- Always compare against a simple baseline.

A.5 Suggested next learning topics

After this handbook, the most useful next topics are: model interpretability, data pipelines with `tf.data`, mixed-precision training, hyperparameter optimization, self-supervised learning, object detection/segmentation, embeddings and vector search, modern LLM fine-tuning methods, and MLOps/model monitoring.

Final self-check

- ☐ Can I explain forward propagation and backpropagation without code?
- ☐ Can I choose the correct output activation and loss for a task?
- ☐ Can I identify overfitting from training/validation curves?
- ☐ Can I explain the difference between Adam and SGD at a high level?
- ☐ Can I explain what convolution, recurrence and attention are designed to exploit?
- ☐ Can I load a saved model and run live inference without retraining?
- ☐ Can I explain why test data should not be used for tuning?
- ☐ Can I build a small model, evaluate it with appropriate metrics and debug common failures?

If you can answer most of these confidently: You have moved beyond beginner-level Deep Learning and have a practical intermediate foundation. The next step is repeated project work, deeper mathematics, and reading architecture-specific documentation/papers.